



THE UNIVERSITY
OF QUEENSLAND
A U S T R A L I A

Inferring security-relevant properties from Java documentation

Connor Myers

Bachelor of Engineers (Honours)

Abstract

Start this section on a new page [this template will automatically handle this].

The abstract should outline the main approach and findings of the thesis and normally must be between 300 and 800 words.

Declaration by author

(All candidates to reproduce this section in their thesis verbatim)

This thesis is composed of my original work, and contains no material previously published or written by another person except where due reference has been made in the text. I have clearly stated the contribution by others to jointly-authored works that I have included in my thesis.

I have clearly stated the contribution of others to my thesis as a whole, including statistical assistance, survey design, data analysis, significant technical procedures, professional editorial advice, financial support and any other original research work used or reported in my thesis. The content of my thesis is the result of work I have carried out since the commencement of my higher degree by research candidature and does not include a substantial part of work that has been submitted to qualify for the award of any other degree or diploma in any university or other tertiary institution. I have clearly stated which parts of my thesis, if any, have been submitted to qualify for another award.

I acknowledge that an electronic copy of my thesis must be lodged with the University Library and, subject to the policy and procedures of The University of Queensland, the thesis be made available for research and study in accordance with the Copyright Act 1968 unless a period of embargo has been approved by the Dean of the Graduate School.

I acknowledge that copyright of all material contained in my thesis resides with the copyright holder(s) of that material. Where appropriate I have obtained copyright permission from the copyright holder to reproduce material in this thesis and have sought permission from co-authors for any jointly authored works included in the thesis.

Chapter 1

Introduction

Static application security testing (SAST) is the application of static program analysis to the security of software projects. [MORE INFO ABOUT STATIC ANALYSIS HERE](#). Static analysis is performed on the source code, compiled byte-code, or some other representation of the source code [1]. Popular SAST tools include: SonarQube [2], SpotBugs [3], Snyk Code [4], and Facebook’s Infer [5]. SAST tools have become widely used at all stages of the software development life-cycle. Furthermore, they have been demonstrated to be highly effective in identifying defects in code compared to other methods, such as manual inspection [6].

Unfortunately, SAST tools are not perfect. From Rice’s Theorem, it is known that any non-trivial property about programs is reducible to the halting problem. That is, it is undecidable — a “perfect” analysis is not always possible [7]. As such, SAST tools are required to make approximations, resulting in an imperfect output — i.e. false positives and false negatives [8]. False positives are not ideal as manual inspection is required to separate genuine defects from false positives, leading to increased time and labour cost. And, to reduce the rate of false negatives, manual configuration of the SAST tools for specific software projects is often required [9].

For example, to perform an effective taint analysis a custom configuration of sources, sinks and sanitisers for a particular project is often required when the default configuration proves insufficient. Creating this manual configuration is a time consuming and resource intensive process. As such, methodologies automatically labelling methods as sources, sinks, and sanitisers may enable existing taint analysis tools to operate more effectively and efficiently. It should also be noted that in cases where a closed-source library is being used in a software project, it may be impossible to determine which methods in the API should be manually added to the taint analysis.

Deep learning, a subtopic within the broader machine learning, has seen an astounding rise in popularity in recent years. What was once a niche topic reserved for academics with access to computational resources far out of reach by most has turned into arguably one of the most influential technologies of

the modern age. Deep learning has fundamentally changed our modern life, with applications ranging from targeted advertising to identifying COVID-19 cases from lung scans [10]. One often over-looked application of deep-learning, lost amidst hype concerning self-driving cars and ... is natural-language processing (NLP).

NLP has existed in one form or another for decades. In the early days of NLP, work was painstakingly done using handwritten rules before moving into probabilistic methods in the 1990s. Naturally, NLP was completely revolutionised with the rising popularity of deep-learning [11]. Furthermore, with companies such as Google and Microsoft publicly releasing large pre-trained models, NLP has never been more accessible. Popular applications of NLP (using deep-learning) is machine translation and speech recognition. One application of deep-learning, and NLP, that has not been explored thoroughly in the literature is its application to software security.

Javadoc, while being an informal description of code, may contain information relevant to software security. With powerful pre-trained models such as CodeBERT, which “captures the semantic connection between natural language and programming language, and produces general-purpose representations that can broadly support NL-PL understanding tasks” [12], it may be possible to fine-tune CodeBERT to extract security relevant properties from Javadocs in order to configure existing SAST tools.

1.1 Topic Definition

Introduce your topic.

Chapter 2

Full title

Introduce the broad layout of the chapter.

2.1 Introduction

Add your text here.

Chapter 3

Conclusion

Conclude your thesis.

Bibliography

- [1] F. Nielson, H. R. Nielson, C. Hankin, Principles of program analysis / Flemming Nielson, Hanne Riis Nielson, Chris Hankin., 1st Edition, Springer-Verlag, Berlin ; Heidelberg ; New York, 2005.
- [2] SonarSource, Sonarqube (2022).
URL <https://www.sonarqube.org/>
- [3] Spotbugs (2022).
URL <https://spotbugs.github.io/>
- [4] Snyk, Snykcode (2022).
URL <https://snyk.io/>
- [5] Facebook, Infer (2022).
URL <https://fbinfer.com/>
- [6] W. R. Nichols, The cost and benefits of static analysis during development (2020). doi:10.48550/ARXIV.2003.03001.
URL <https://arxiv.org/abs/2003.03001>
- [7] A. Møller, M. I. Schwartzbach, Static program analysis, Notes. Feb (2012).
- [8] B. Chess, G. McGraw, Static analysis for security, IEEE security & privacy 2 (6) (2004) 76–79.
- [9] F. Zampetti, S. Scalabrino, R. Oliveto, G. Canfora, M. Di Penta, How open source projects use static code analysis tools in continuous integration pipelines, in: 2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR), IEEE, 2017, pp. 334–344.
- [10] M. Rahimzadeh, A. Attar, S. M. Sakhaei, A fully automated deep learning-based network for detecting covid-19 from a new and large lung ct scan dataset, Biomedical Signal Processing and Control 68 (2021) 102588. doi:<https://doi.org/10.1016/j.bspc.2021.102588>.
URL <https://www.sciencedirect.com/science/article/pii/S1746809421001853>
- [11] L. Deng, Y. Liu, Deep learning in natural language processing / Li Deng, Yang Liu, editors., Springer, Singapore, 2018.

- [12] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, M. Zhou, Codebert: A pre-trained model for programming and natural languages (2020). doi: 10.48550/ARXIV.2002.08155.
URL <https://arxiv.org/abs/2002.08155>

Appendix A

Appendix

Write your appendix here.